

COURSE 2B - DA-to-DE Bridge

B04 Warehousing & Data Quality Foundations

From analyst outputs to reliable analytical-system reasoning

Fast-track purpose

Course 1 taught you to clean, model and validate analytical data. B04 does not turn those analyst skills into a second BI course. It teaches why engineers separate workloads, preserve raw evidence, design history, define layers, choose latency, track lineage, enforce quality contracts and recover safely before C3 implements those systems.

Locked learning loop

Why -> picture -> new words -> watch -> analyst-to-engineer transition -> follow once -> observe -> explain -> mistakes -> recover -> fresh task -> validate -> teach back -> protected review -> changed retry -> evidence/route.

INTERNAL QA RC1 - not Course 2B FINAL

Contents & Study Route

Lesson	Title	Pages
B04-L01	Operational vs analytical workloads: OLTP and OLAP	3-4
B04-L02	Warehouse, lake and lakehouse mental models	5-6
B04-L03	Business keys, surrogate keys and SCD history	7-8
B04-L04	ETL vs ELT and transformation boundaries	9-10
B04-L05	Layered analytical systems: raw, staged, conformed and serving	11-12
B04-L06	Batch, micro-batch and streaming trade-offs	13-14
B04-L07	Lineage, metadata, ownership and observability basics	15-16
B04-L08	Data-quality contracts: block, warn and reconcile	17-18
B04-L09	Replay, backfill and warehouse failure-recovery reasoning	19-20
Quick reference	CareFlow Mini-Lab / Gate route / C3 boundary	21

Exact module route

For each lesson: learner book -> follow once -> matching workbook sheet -> save genuine attempt/evidence -> protected review -> close review -> changed retry. After B04-L09: complete the fresh CareFlow Clinic Warehouse Mini-Lab from a clean copy -> save evidence -> Mini-Lab protected review -> Gate A. Gate B/C remain sealed unless a full reassessment is explicitly assigned.

Course 3 boundary

B04 teaches the architecture and failure reasoning C3 depends on. It does NOT implement dbt/materializations/SCD jobs (C3-02), Kafka/stream processing (C3-06), or require a Fabric tenant. C3-00 still teaches Linux/Git/venv/testing/container foundations from zero.

B04-L01 - Operational vs analytical workloads: OLTP and OLAP

1. Why it matters

A transactional database is optimized to record business events safely. An analytical system is optimized to scan, combine and summarize history. Mixing those jobs carelessly can slow operations and still produce poor analytics.

2. What you are learning

Classify a workload as operational or analytical, then write the boundary that protects the source while enabling repeatable analysis.

3. Prerequisite

Course 1 SQL and business-analysis skills. You only need to recognize tables, rows, keys and business questions.

4. Picture it in your head

Picture a shop counter and a back-office planning room. The counter must record each sale quickly and correctly. The planning room asks what happened across millions of sales. Same business, different workload.

5. New words before use

OLTP	online transaction processing; many small reads/writes that run the business
OLAP	online analytical processing; larger scans/aggregations used to understand the business
source system	the operational system where business events are first recorded
analytical workload	queries designed for history, comparison, aggregation and decision support

6. Watch / visual source

Optional visual: Alex The Analyst, Database vs Data Warehouse vs Data Lake, 00:00-05:22. After watching, write one OLTP and one OLAP example from the same business. Current reference: [Microsoft Fabric Data Warehouse](#).

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already know how to query tables and answer business questions.

What changes in engineering: In engineering, you must protect the operational source and decide where analytical work should run.

Why engineers care: Heavy scans or uncontrolled transformations on a production source can hurt the system customers depend on.

Small worked example: Ride booking INSERT/update is OLTP. Monthly city revenue by driver cohort is OLAP.

Real engineering version: C3 will build bounded extraction and landing patterns; this lesson only establishes why the boundary exists.

7. Follow with me once

Open B04_WAREHOUSING_DATA_QUALITY_PRACTICE_RC1.xlsx -> Workload_Contract. Classify six Urban Mobility activities, then write one source-protection rule for each analytical request.

8. What you should see

Operational actions favor small, current, keyed transactions. Analytical requests favor historical scans, joins, aggregations and separate compute/storage boundaries.

B04-L01 - Practice to mastery

9. Why it worked

The classification starts from purpose and access pattern, not the product name. The same company can legitimately need both workloads.

10. Common beginner mistakes

Calling every SQL database a warehouse; running dashboard-style full scans directly on the app database; treating OLTP/OLAP as vendor brands.

11. If stuck

Ask: is this action recording/changing one business event now, or summarizing many events to make a decision? Then identify who is harmed if the query is slow.

12. Small version alone

For a clinic, classify appointment booking, payment capture, weekly no-show analysis and annual provider-capacity planning.

13. Validate

Every row has workload type + reason + source-risk note. At least one analytical request has an explicit "do not run directly on operational source" boundary.

14. Teach it back

Explain OLTP vs OLAP in 60-90 seconds without using the words "small" and "big" as your only reason.

15. Protected review + changed retry

Save the attempt first. Open Review B04-L01 only afterward. Close it and redo the classification for a food-delivery business with changed activities.

16. Evidence / mastery + next route

Saved Workload_Contract attempt + changed-domain retry + source-protection rule + own-words explanation. PASS -> B04-L02; otherwise targeted workload-boundary repair.

B04-L02 - Warehouse, lake and lakehouse mental models

1. Why it matters

Engineers choose storage/serving patterns by workload, structure, governance and consumers. Memorizing three definitions is not enough; a poor fit can create expensive duplication or an ungoverned data swamp.

2. What you are learning

Choose among warehouse, lake and lakehouse mental models for a stated need and defend one rejected alternative.

3. Prerequisite

B04-L01. No cloud account or platform implementation is required.

4. Picture it in your head

Imagine three storage rooms: a labeled reporting room, a flexible raw-material warehouse, and a flexible warehouse with a managed catalog/table layer on top. The point is not which is "best"; it is which contract fits the work.

5. New words before use

data warehouse	structured analytical store optimized for governed SQL/BI workloads
data lake	low-cost flexible storage for raw/varied data, usually files/object storage
lakehouse	lake-style storage plus table/management capabilities for reliable analytics
schema-on-write	structure is enforced before/while writing
schema-on-read	structure is interpreted when data is read

6. Watch / visual source

Book-led visual in this lesson: use the three-column decision matrix. Current references: Fabric Data Warehouse and IBM comparison. The prior IBM video remains audit-history backup only because an exact useful segment was not independently verified.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already know structured tables are convenient for BI.

What changes in engineering: Engineering adds raw-data retention, varied formats, governance, cost and multi-workload decisions.

Why engineers care: A reporting-friendly schema and a raw-history store solve different problems; forcing one to do everything creates fragility.

Small worked example: Curated monthly sales mart -> warehouse fit. Raw JSON clickstream archive -> lake fit. Unified table layer over object storage for BI + engineering -> lakehouse fit.

Real engineering version: C3-04 later implements lakehouse/table-format concepts. B04 only teaches the decision model.

7. Follow with me once

Open Storage_Fit. For each Urban Mobility scenario, choose warehouse/lake/lakehouse and state structure, consumer, governance and replay needs before choosing.

8. What you should see

Different scenarios can reasonably choose different patterns. Your decision cites requirements, not popularity.

B04-L02 - Practice to mastery

9. Why it worked

The decision matrix forces evidence: data shape + access pattern + governance + cost/replay. This prevents "lakehouse because modern" reasoning.

10. Common beginner mistakes

Calling a folder of CSVs a governed lakehouse; assuming a lake is automatically cheaper overall; choosing a warehouse for raw binary/unstructured archives without a reason.

11. If stuck

Write four questions: What data shape? Who queries it? How much governance/transaction reliability? Must raw history remain cheaply replayable?

12. Small version alone

Choose storage fits for: raw IoT logs, finance reporting mart, and mixed ML + BI history. Reject one alternative for each.

13. Validate

Every decision names at least three requirements and one trade-off; one rejected alternative is defended.

14. Teach it back

Explain why lakehouse is not simply "warehouse + lake" as a marketing slogan.

15. Protected review + changed retry

Attempt -> protected Review B04-L02 -> close review -> redo with an e-commerce scenario containing images, orders and governed finance metrics.

16. Evidence / mastery + next route

Storage_Fit decisions + rejected alternatives + changed retry + own-words explanation. PASS -> B04-L03.

B04-L03 - Business keys, surrogate keys and SCD history

1. Why it matters

Analysts often receive a clean dimension table. Engineers must decide how source identities and changing descriptive history will be represented without rewriting the past.

2. What you are learning

Distinguish business and surrogate keys, choose Type 1 vs Type 2 history behavior, and define effective-date rules at a conceptual level.

3. Prerequisite

B03 grain/key contracts. No SCD implementation code is expected yet.

4. Picture it in your head

A provider keeps the same employee number but changes clinic and specialty over time. The source identity is one person; the warehouse may need multiple historical versions so an old appointment still sees the old context.

5. New words before use

business key	identifier supplied by the source/business, such as <code>provider_id</code>
surrogate key	warehouse-generated identifier for a dimension row/version
SCD	slowly changing dimension; a pattern for handling descriptive changes
Type 1	overwrite current descriptive value; no history retained
Type 2	create a new dimension version and preserve prior history
effective dating	start/end rules describing when a version is valid

6. Watch / visual source

Official references: Dimensional modeling overview and Load tables in a dimensional model. Book-led worked timeline is the visual for this lesson.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already know a dimension needs a usable key.

What changes in engineering: Engineering decides what identity comes from the source, what warehouse key represents a version, and whether a change rewrites or preserves history.

Why engineers care: Without history rules, last year's facts can suddenly inherit today's customer/provider attributes.

Small worked example: `provider_id=P07` remains the business key. City correction typo -> Type 1. Provider changes clinic on 2026-04-01 and history matters -> Type 2 version with a new surrogate key.

Real engineering version: C3-02 later implements snapshots/SCD and late-dimension behavior. Here you must be able to design and defend the rule.

7. Follow with me once

Open `History_Keys`. Inspect `driver_profile_changes`. Choose Type 1/Type 2 behavior for each attribute, then write a conceptual surrogate/effective-date strategy and the rule for duplicate effective-date rows.

8. What you should see

You separate identity from version. A Type 2 history table can contain multiple rows for one business key but never ambiguous overlapping active versions under the declared rule.

B04-L03 - Practice to mastery

9. Why it worked

The timeline makes historical truth explicit. Facts can later resolve to the correct version by event date rather than today's description.

10. Common beginner mistakes

Using a business key as if it uniquely identifies every historical version; Type 2 for every typo; overlapping effective intervals; silently deleting duplicate change rows.

11. If stuck

Ask whether the old value must remain historically true. If yes, history likely matters. Then state the source identity separately from the warehouse version identity.

12. Small version alone

Design history for a customer segment change, corrected spelling, and store-region reassignment.

13. Validate

Business key stated; history choice justified per attribute; no ambiguous active interval; duplicate change trap is surfaced rather than hidden.

14. Teach it back

Explain why a surrogate key does not replace the business key's business meaning.

15. Protected review + changed retry

Attempt -> Review B04-L03 -> close -> changed clinic provider-history case with different effective dates.

16. Evidence / mastery + next route

History_Keys design + one Type 1 and one Type 2 justification + ambiguity check + changed retry. PASS -> B04-L04.

B04-L04 - ETL vs ELT and transformation boundaries

1. Why it matters

ETL and ELT differ mainly in where durable raw data lands relative to transformation. That boundary affects replay, governance, compute, debugging and how safely rules can change.

2. What you are learning

Draw ETL, ELT and hybrid flows and choose where validation/transformation should occur for a stated constraint.

3. Prerequisite

B04-L01-L03. You do not need dbt, orchestration or cloud implementation yet.

4. Picture it in your head

ETL is inspect/prepare before the main destination. ELT is land first, then transform inside/near the analytical platform. A hybrid can still block unsafe fields before landing while retaining replayable raw data where allowed.

5. New words before use

extract	read/copy data from the source
transform	clean, standardize, join, derive or reshape data
load	write data to the target
landing	durable early copy of source data/metadata
transformation boundary	the place where business/quality logic is allowed to change data

6. Watch / visual source

Optional visual: Kahan Data Solutions, ETL vs ELT - Modern Data Architectures, 00:36-03:19 (ETL 00:36; ELT 01:38; comparison/trade-off begins 03:19). After watching: close it, draw both three-step arrows, and mark where raw data first becomes durable. Official reference: [Microsoft load-tables guidance](#).

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already transform data in Power Query, SQL or pandas.

What changes in engineering: Engineering chooses the durable landing point, transformation boundary and recovery contract around those transformations.

Why engineers care: If a bad transformation overwrites the only copy, replay becomes difficult or impossible.

Small worked example: ELT: source -> raw landing -> warehouse transformation -> mart. ETL: source -> controlled transform -> curated warehouse. Hybrid: sensitive-field validation/redaction before durable landing plus later analytical transformations.

Real engineering version: C3-01 builds landing/replay; C3-02 implements transformation layers and dbt. B04 only makes the boundary reasoning explicit.

7. Follow with me once

Open Transform_Boundaries. Draw ETL and ELT for rides + payments. Choose one hybrid constraint and state which data remains replayable.

8. What you should see

Your diagram distinguishes source, durable landing/staging, transformation step and curated consumer output.

B04-L04 - Practice to mastery

9. Why it worked

A named boundary separates raw evidence from business logic. That makes change/recovery reasoning possible.

10. Common beginner mistakes

Saying ETL means "no cleaning"; calling every sequence ETL; transforming raw evidence in place; assuming cloud always means ELT.

11. If stuck

Mark the first durable copy. Then ask: if tomorrow's rule changes, what can I replay without requesting the source again?

12. Small version alone

Design ETL/ELT/hybrid for daily banking transactions with one sensitive field and a monthly reporting mart.

13. Validate

Flow has explicit source, durable point, transform boundary, curated output and replay statement.

14. Teach it back

Explain ETL vs ELT using where transformation happens relative to durable load, not just the letter order.

15. Protected review + changed retry

Attempt -> Review B04-L04 -> close -> changed subscription-billing flow.

16. Evidence / mastery + next route

Two flow diagrams + one chosen architecture + replay rationale + changed retry. PASS -> B04-L05.

B04-L05 - Layered analytical systems: raw, staged, conformed and serving

1. Why it matters

Layers are useful only when each has a clear contract. Naming folders Bronze/Silver/Gold without defining allowed transformations, consumers and quality gates creates decorative architecture rather than reliability.

2. What you are learning

Define raw, staged/conformed and serving layer contracts with allowed changes, consumer expectations and quality gates.

3. Prerequisite

B04-L04. No Fabric tenant is required.

4. Picture it in your head

Think of evidence -> preparation -> trusted product. Raw preserves what arrived. Staged/conformed normalizes and integrates. Serving/gold exposes stable business-ready models to consumers.

5. New words before use

raw/bronze	source-faithful landed data plus ingestion metadata
staged/silver	typed, normalized, quality-aware integrated data
conformed	shared definitions/keys used consistently across subject areas
serving/gold	curated consumer-facing tables/models
layer contract	what may enter, what transformations are allowed, who consumes it, and what quality must pass

6. Watch / visual source

Optional visual: Microsoft Learn Together, Organize a Fabric lakehouse using medallion architecture, 10:09-30:32. After watching, [define allowed state + consumer + quality gate for raw/silver/gold](#). Current reference: [Fabric medallion architecture](#).

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already clean raw data and publish a report-ready result.

What changes in engineering: Engineering makes intermediate states explicit, reproducible and owned instead of hiding all steps inside one workbook/notebook.

Why engineers care: Clear layer contracts make debugging, lineage, testing and replay localizable.

Small worked example: Raw rides keeps source columns + load metadata. Staged rides types timestamps and flags invalid keys. Conformed aligns rider/driver definitions. Serving exposes monthly_city_metrics at its declared grain.

Real engineering version: C3 later implements these layers with actual files/tables/models; B04 teaches the contract each layer must satisfy.

7. Follow with me once

Open Layer_Contracts. Place each Urban Mobility source and derived output into a layer. For each layer write allowed changes, quality gate and consumer.

8. What you should see

Raw stays recoverable; quality problems can be quarantined/flagged without pretending they never existed; serving outputs have declared grains and definitions.

B04-L05 - Practice to mastery

9. Why it worked

Each transformation belongs to a named stage with an owner and validation point, so failures can be isolated.

10. Common beginner mistakes

Editing raw source rows; putting business KPIs into raw; using layer names with no consumer/quality contract; assuming every project needs exactly three physical layers.

11. If stuck

For each layer answer: what can enter, what may change, who trusts it, and what blocks promotion?

12. Small version alone

Define layers for support tickets + ticket events + daily team SLA metrics.

13. Validate

Every layer has allowed state, transformation boundary, consumer, quality gate and recoverability statement.

14. Teach it back

Explain why "Bronze/Silver/Gold" labels alone do not create a medallion architecture.

15. Protected review + changed retry

Attempt -> Review B04-L05 -> close -> changed logistics flow.

16. Evidence / mastery + next route

Layer_Contracts map + one rejected misplaced transformation + changed retry. PASS -> B04-L06.

B04-L06 - Batch, micro-batch and streaming trade-offs

1. Why it matters

Faster is not automatically better. Latency choice changes complexity, cost, ordering, late-data handling, replay and operational expectations.

2. What you are learning

Choose batch, micro-batch or streaming from business SLA and failure/replay needs, not from tool popularity.

3. Prerequisite

B04-L05. Kafka or stream-processing code is explicitly not required.

4. Picture it in your head

Batch is a scheduled truck carrying many records. Micro-batch is a smaller truck every few minutes. Streaming is a conveyor belt of events. All can fail; each needs a replay strategy.

5. New words before use

latency	delay between an event occurring and being available to the consumer
batch	process a bounded collection on a schedule
micro-batch	process small bounded groups frequently
streaming	continuously process events as they arrive
late data	events arriving after the expected processing window
SLA	agreed service expectation, such as freshness or recovery time

6. Watch / visual source

Book-led latency/replay decision matrix. Current reference: Fabric data movement decision guide. No external video is required for this lesson.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already know a dashboard can refresh daily or more often.

What changes in engineering: Engineering must connect freshness requirements to ingestion mode, event ordering, state, cost and recovery complexity.

Why engineers care: Unnecessary real-time design adds failure modes; insufficient freshness can make a system useless.

Small worked example: Daily finance close -> batch. Five-minute operations board -> micro-batch. Fraud/event response requiring seconds -> streaming candidate.

Real engineering version: C3-06 later teaches Kafka/streaming implementation; B04 only establishes when that complexity is justified.

7. Follow with me once

Open Latency_Plan. For four Urban Mobility consumers, write required freshness, volume/burst, late-data tolerance, replay need and recommended mode.

8. What you should see

At least one case stays batch because its SLA does not justify streaming complexity.

B04-L06 - Practice to mastery

9. Why it worked

The business SLA sets the target. Architecture follows the target plus failure/recovery needs.

10. Common beginner mistakes

Choosing streaming because it sounds advanced; confusing refresh frequency with source event time; ignoring late data; promising zero latency.

11. If stuck

Start with "How stale can this safely be?" Then ask what happens if processing stops for 20 minutes.

12. Small version alone

Choose modes for payroll, ride-location alerts and hourly marketplace inventory.

13. Validate

Each mode has SLA + late-data + replay + cost/complexity rationale.

14. Teach it back

Explain a case where batch is more professional than streaming.

15. Protected review + changed retry

Attempt -> Review B04-L06 -> close -> changed hospital operations scenario.

16. Evidence / mastery + next route

Latency_Plan + one explicit non-streaming decision + changed retry. PASS -> B04-L07.

B04-L07 - Lineage, metadata, ownership and observability basics

1. Why it matters

When a number is wrong, an engineer must know where it came from, which step changed it, who owns the issue and what evidence exists from the run.

2. What you are learning

Create a simple lineage chain and minimum run metadata/ownership contract that supports impact analysis and diagnosis.

3. Prerequisite

B04-L05-L06. No observability platform is required.

4. Picture it in your head

Lineage is a map from source to consumer. Metadata is the labels and run evidence on that map. Ownership tells you who can act when a road is blocked.

5. New words before use

lineage	upstream/downstream dependency map
metadata	data about data/run state, such as source, schema, timestamp and row count
run ID	identifier tying logs/evidence to one execution
owner	person/team accountable for a dataset or system boundary
observability	ability to infer system health from evidence such as freshness, volume, errors and logs

6. Watch / visual source

Current reference: [Microsoft Fabric lineage](#). Book-led lineage diagram is sufficient: no external video is required.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already cite sources and validate outputs.

What changes in engineering: Engineering records dependencies and run evidence systematically so another person can trace impact and failure without guessing.

Why engineers care: Without lineage and ownership, a bad source column can silently affect many downstream tables/reports and nobody knows who should repair it.

Small worked example: rides.csv -> raw_rides -> staged_rides -> monthly_city_metrics -> operations dashboard. Run metadata stores run_id, source file, ingest time, row count, schema version, status and error count.

Real engineering version: C3-02/C3-08 later implement documentation, tests and observability patterns. B04 defines the minimum evidence contract.

7. Follow with me once

Open Lineage_Ownership. Draw one Urban Mobility lineage chain and fill owner, SLA, run metadata and impact response for each node.

8. What you should see

You can answer "what upstream input could have caused this?" and "what downstream consumers are affected?" without opening every file.

B04-L07 - Practice to mastery

9. Why it worked

Dependencies plus run evidence convert debugging from memory into traceable investigation.

10. Common beginner mistakes

Treating a folder path as complete lineage; collecting logs with no run ID; naming an owner with no responsibility boundary; tracking only failures and not normal row/freshness evidence.

11. If stuck

Start from the wrong output and walk upstream one transformation at a time. Write the minimum evidence you would need at each hop.

12. Small version alone

Build lineage for clinic appointments -> cleaned appointments -> provider utilization -> dashboard.

13. Validate

Chain has upstream/downstream links, owner, freshness expectation, run ID evidence and one failure-impact route.

14. Teach it back

Explain the difference between lineage and logging.

15. Protected review + changed retry

Attempt -> Review B04-L07 -> close -> changed subscription pipeline lineage.

16. Evidence / mastery + next route

Lineage diagram + metadata contract + ownership/impact note + changed retry. PASS -> B04-L08.

B04-L08 - Data-quality contracts: block, warn and reconcile

1. Why it matters

Data quality is not "clean everything." Engineers decide which failures block publication, which can be warned/quarantined, what gets reconciled and who owns the decision.

2. What you are learning

Write a data-quality contract with rule, severity, action, owner, evidence and independent reconciliation.

3. Prerequisite

B04-L03 and B04-L07. You already know analyst-side quality checks from Course 1.

4. Picture it in your head

Imagine airport security: some conditions stop the passenger, some trigger secondary screening, some are recorded for follow-up. The action depends on risk, not whether the check is easy.

5. New words before use

quality contract	declared rule + expected result + severity + action + owner + evidence
block	stop promotion/publish because correctness is unsafe
warn	continue with visible risk/evidence
quarantine	separate suspect records for investigation
reconciliation	independent comparison proving counts/totals/keys across boundaries
critical failure	defect that invalidates trustworthy use until resolved

6. Watch / visual source

Current reference: [Microsoft dimensional load guidance](#). Book-led severity matrix is the primary visual.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already check duplicates, nulls, types, keys and totals.

What changes in engineering: Engineering turns those checks into reusable contracts with severity, pipeline action, ownership and auditable evidence.

Why engineers care: A check that finds a problem but does not define what happens next is not an operational control.

Small worked example: Duplicate driver history effective-date pair -> BLOCK history promotion. One orphan event with preserved raw evidence -> QUARANTINE + WARN depending consumer contract. Payment total mismatch -> BLOCK serving publication until reconciled.

Real engineering version: C3-01/C3-02/C3-08 later implement automated tests/quarantine/observability. B04 establishes the decision contract first.

7. Follow with me once

Open `Quality_Contract`. Define six Urban Mobility checks spanning required key, duplicate history, orphan FK, type/business rule, freshness and monetary reconciliation.

8. What you should see

Every check has expected result, severity, action, owner and saved evidence. Not every defect is silently dropped.

B04-L08 - Practice to mastery

9. Why it worked

Severity links technical evidence to business risk. Independent reconciliation catches failures that a single row-level rule can miss.

10. Common beginner mistakes

Replacing errors with zero; deleting orphans to make counts match; marking every issue BLOCK; using one grand total as proof of all correctness.

11. If stuck

For each defect ask: can a consumer make a materially wrong decision if this passes? If yes, block or isolate until explicitly resolved.

12. Small version alone

Write quality contracts for clinic appointment IDs, payment totals, provider history and late events.

13. Validate

At least one BLOCK, one WARN/QUARANTINE, one independent reconciliation and named owner. No issue is hidden by deletion.

14. Teach it back

Explain why "0 errors" can be less trustworthy than an explicit quarantine count.

15. Protected review + changed retry

Attempt -> Review B04-L08 -> close -> changed marketplace quality case.

16. Evidence / mastery + next route

Quality_Contract + severity rationale + reconciliation + changed retry. PASS -> B04-L09.

B04-L09 - Replay, backfill and warehouse failure-recovery reasoning

1. Why it matters

A professional pipeline is not proven by one successful run. You need to know how to recover from bad logic, missing periods or partial failure without corrupting trusted outputs.

2. What you are learning

Write a conceptual replay/backfill runbook that preserves raw evidence, isolates scope, protects live state and proves correctness after recovery.

3. Prerequisite

B04-L04-L08. Implementation of checkpoints/orchestration belongs to C3.

4. Picture it in your head

Think of a damaged chapter in a printed ledger. You do not rewrite the entire archive from memory. You identify the affected pages, return to preserved evidence, reprocess a bounded range, compare controls, and only then republish.

5. New words before use

replay	run processing again from preserved input/state
backfill	process a historical period that was missed or needs rebuilding
idempotence	rerunning safely produces the intended state without accidental duplication
checkpoint	record of processing progress/state
bounded recovery	reprocess only a declared scope/window with controlled state
post-recovery validation	independent proof the repaired result is complete and not duplicated

6. Watch / visual source

Book-led recovery sequence. Current reference: Fabric medallion architecture for preserved raw/layered reasoning. No external video is required.

Bridge transition - Analyst -> Engineer

What you already know as an analyst: You already rerun a notebook or refresh a report when something changes.

What changes in engineering: Engineering makes rerun scope, durable inputs, state protection, duplicate prevention and post-recovery proof explicit.

Why engineers care: Blind reruns can double-count facts, overwrite history or advance state before completeness is proven.

Small worked example: Bad transformation affected rides for 2026-03-01. Freeze serving output, identify affected window, replay from preserved raw, write to isolated candidate, reconcile rows/payment total, compare keys, then promote and record run evidence.

Real engineering version: C3-01 implements state/replay/backfill and C3-00/C3-03 deepen idempotence/runbooks/orchestration. B04 must prepare the reasoning without hiding those prerequisites.

7. Follow with me once

Open Recovery_Runbook. Use an Urban Mobility failure scenario: one day of rides loaded but payment reconciliation failed. Write detect -> contain -> scope -> replay -> validate -> promote -> document.

8. What you should see

State is not advanced simply because part of the run succeeded. Raw evidence stays unchanged. Promotion occurs only after independent controls pass.

B04-L09 - Practice to mastery

9. Why it worked

Bounded recovery separates diagnosis, reprocessing and promotion. It prevents "rerun everything and hope" behavior.

10. Common beginner mistakes

Deleting bad output before preserving evidence; advancing checkpoint before reconciliation; rerunning unbounded history; validating only the metric that originally failed.

11. If stuck

Write seven verbs in order: detect, contain, identify scope, recover from preserved input, validate independently, promote, document.

12. Small version alone

Write a backfill plan for two missing days of support-ticket events when daily SLA aggregates already exist.

13. Validate

Runbook names affected window, source evidence, state/checkpoint handling, duplicate prevention, at least two independent controls and promotion/rollback rule.

14. Teach it back

Explain the difference between replay and backfill and why both need post-recovery validation.

15. Protected review + changed retry

Attempt -> Review B04-L09 -> close -> changed CareFlow payment/orphan-event recovery case.

16. Evidence / mastery + next route

Recovery runbook + bounded scope + independent controls + changed retry. PASS -> independent CareFlow Mini-Lab; after Mini-Lab PASS -> protected Gate A.

B04 Quick Reference - Mini-Lab, Gates and C3 handoff

Fresh independent Mini-Lab

Open B04_CAREFLOW_WAREHOUSE_MINI_LAB_RC1.xlsx with B04_CAREFLOW_MINILAB_DATA_RC1.zip. Do not use the Urban Mobility practice workbook as a solution template. Save architecture decisions, quality/recovery contract, reconciliation and 60-90 second defense before opening the protected Mini-Lab review.

Gate route

Gate A - Marketplace Sellers: first protected independent Gate. Gate B - Inventory Replenishment: only after explicit full-retest assignment following Gate A weakness/repair. Gate C - Customer Success Renewals: only after explicit second full-retest assignment following Gate B instability. Targeted repair never auto-opens B or C.

Critical pass behavior

A polished architecture diagram cannot pass if grain/history logic, quality severity, reconciliation, lineage/ownership or recovery controls are materially unsafe. Failed attempts remain append-only evidence.

Worksheet index

ID	Practice sheet	Main proof
B04-L01	Workload_Contract	Every row has workload type + reason + source-risk note. At least one analytical request has an explicit "do not run directly on operational source" boundary.
B04-L02	Storage_Fit	Every decision names at least three requirements and one trade-off; one rejected alternative is defended.
B04-L03	History_Keys	Business key stated; history choice justified per attribute; no ambiguous active interval; duplicate change trap is surfaced rather than hidden.
B04-L04	Transform_Boundaries	Flow has explicit source, durable point, transform boundary, curated output and replay statement.
B04-L05	Layer_Contracts	Every layer has allowed state, transformation boundary, consumer, quality gate and recoverability statement.
B04-L06	Latency_Plan	Each mode has SLA + late-data + replay + cost/complexity rationale.
B04-L07	Lineage_Ownership	Chain has upstream/downstream links, owner, freshness expectation, run ID evidence and one failure-impact route.
B04-L08	Quality_Contract	At least one BLOCK, one WARN/QUARANTINE, one independent reconciliation and named owner. No issue is hidden by deletion.
B04-L09	Recovery_Runbook	Runbook names affected window, source evidence, state/checkpoint handling, duplicate prevention, at least two independent controls and promotion/rollback rule.

C3 entry relevance

B04 prepares the reasoning behind C3-01 ingestion/replay, C3-02 transformation/warehouse implementation, C3-06 streaming choices and C3-08 quality/observability. It does not claim those implementation skills are already mastered.

Ready to leave B04 when

All nine lesson changed retries are passed, CareFlow Mini-Lab passes with evidence, and one protected full Gate passes with no unresolved critical failure. Then continue to B05 Cloud Platform Foundations.